

REST API FLASK и сериализация

Содержание урока

- ★ Что такое REST API
- ★ Паттерн контроллер и роутинг
- ★ Flask и с чем его едят
- ★ Сериализация и десериализация

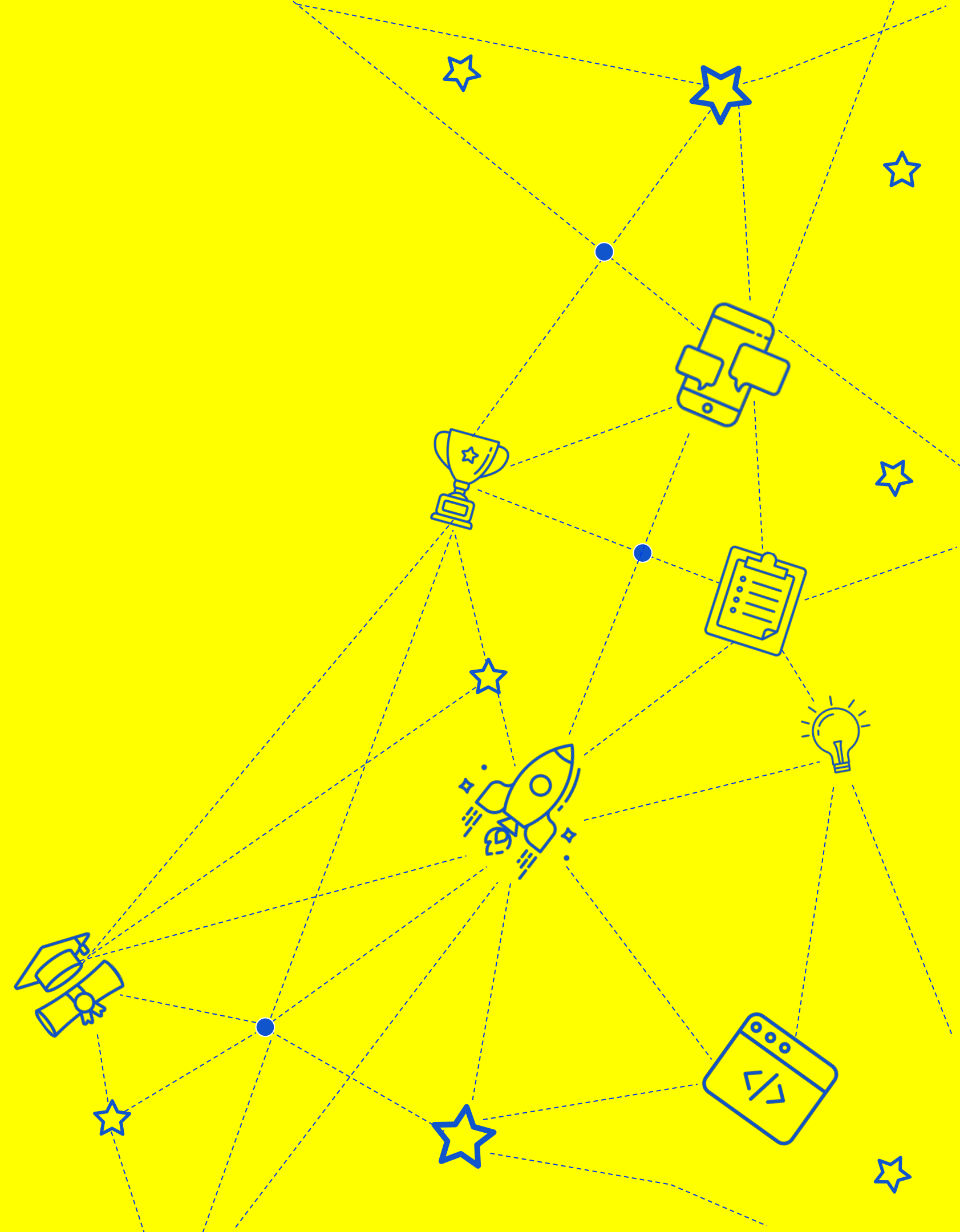


ИГОРЬ КРИВОНОС

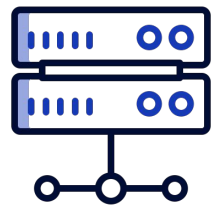
Senior Backend Software Engineer

- Консультирую стартапы по построению масштабируемой архитектуры приложения
- Активно занимаюсь менторингом
- Проектирую и реализовываю backend api с 2014 года

Что такое REST API



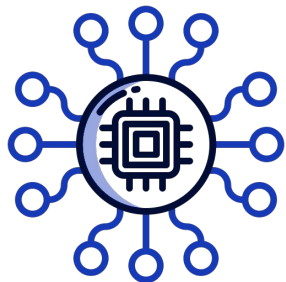
Основные понятия



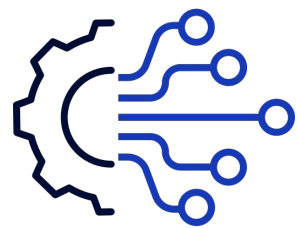
Backend – сервер, который предоставляет доступ к своим функциям посредством API.



Клиент – это пользователь, который использует backend через API, это может быть web, mobile-приложение или другой backend.



Ресурс – бизнес-сущность, которая «живет» внутри backend'а приложения, с которым может взаимодействовать клиент.



REST API – это подход проектирования API с использованием протокола HTTP, при котором клиент взаимодействует с backend'ом через создание, изменения и удаление ресурсов, доступных в приложении.

Ресурс

Может быть представлен любым типом информации:



документ



файл



картинка

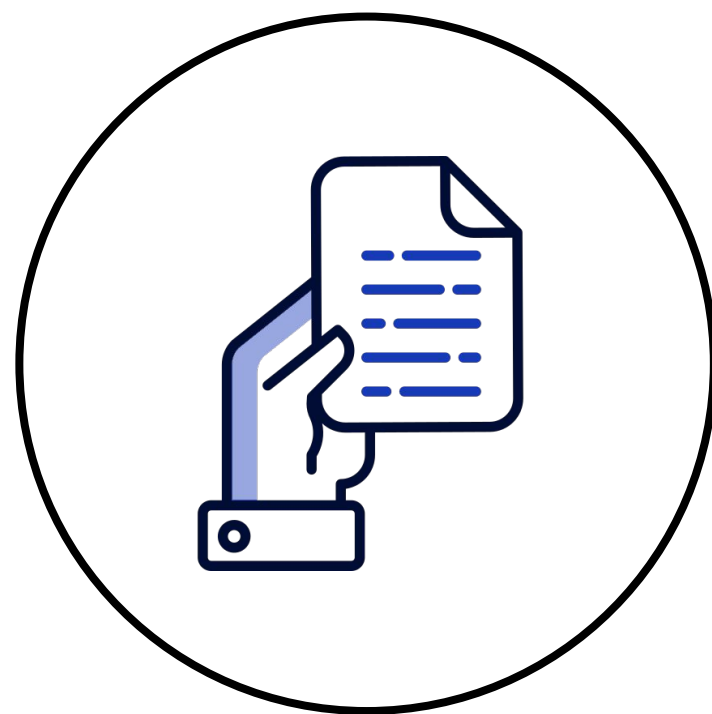
Как работает REST API

Каждый ресурс в REST API имеет уникальный ID, который называется **URI**.

Какие бывают URI:



на все записи



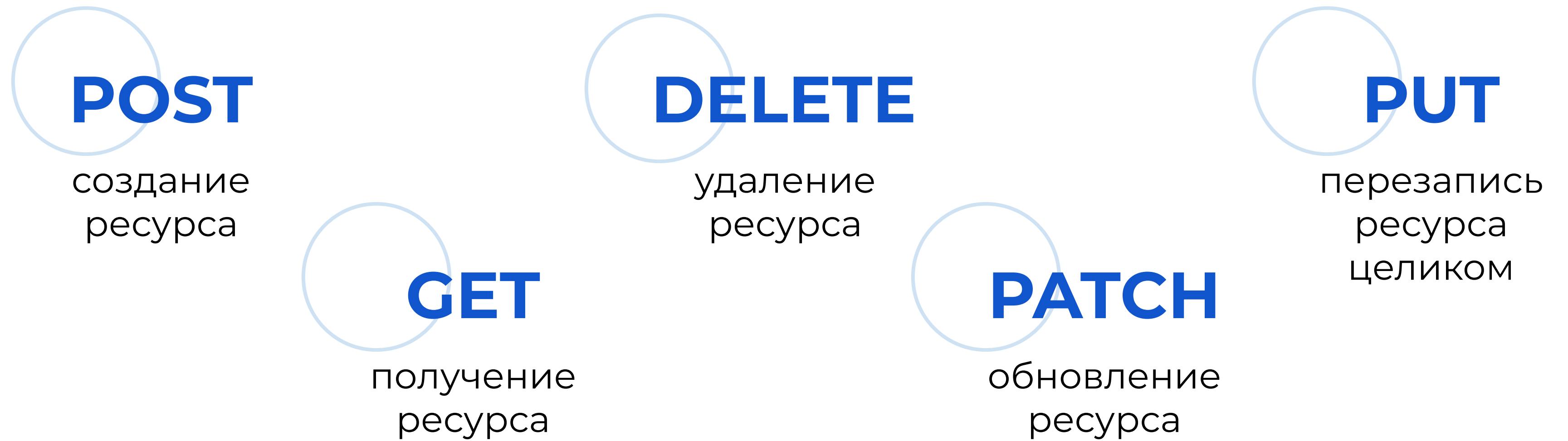
на конкретный
экземпляр



дочерние
экземпляры

Как работает REST API

С ресурсами в REST API можно взаимодействовать при помощи HTTP-запросов:



POST

создание
ресурса

DELETE

удаление
ресурса

PUT

перезапись
ресурса
целиком

GET

получение
ресурса

PATCH

обновление
ресурса

HTTP Codes

При использовании REST API backend должен использовать HTTP коды ответов:



**группа ответов
успешной
обработки**

- ★ **200**, OK
- ★ **201**, Created
- ★ **202**, Accepted



**клиентские
ошибки**

- ★ **400**, Bad Request
- ★ **401**, Not authorized
- ★ **403**, Forbidden
- ★ **404**, Not found



**ошибки
сервера**

- ★ **500**, Internal Server Error
- ★ **503**, Service unavailable

Формат общения

REST API не привязан

ни к какому конкретному формату,

но чаще всего используется **JSON**



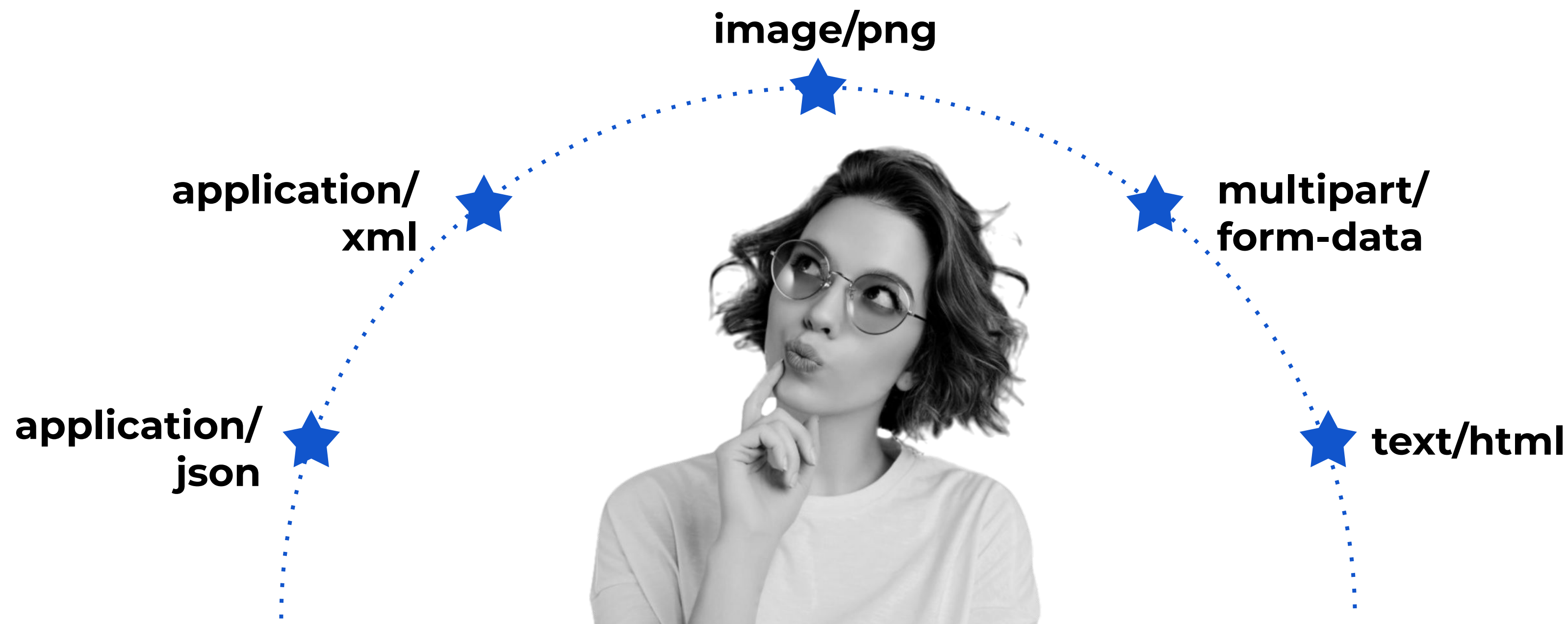
Как понять: что внутри запроса?

Через HTTP Header

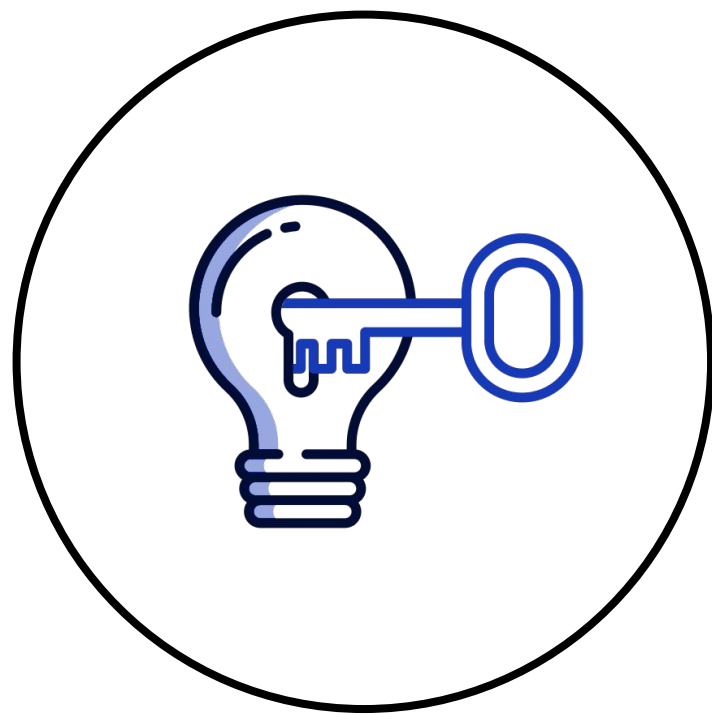


HTTP заголовок Content-Type

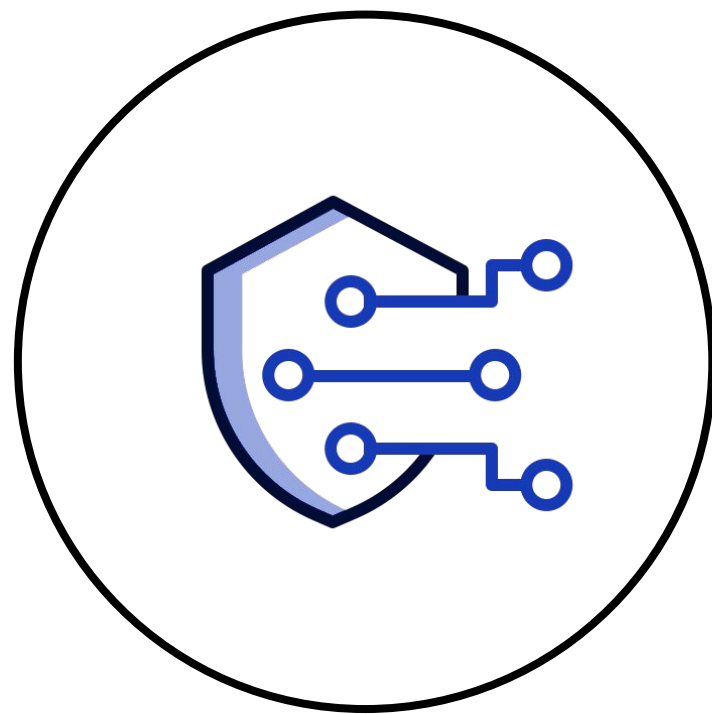
Вы можете встретить следующие типы:



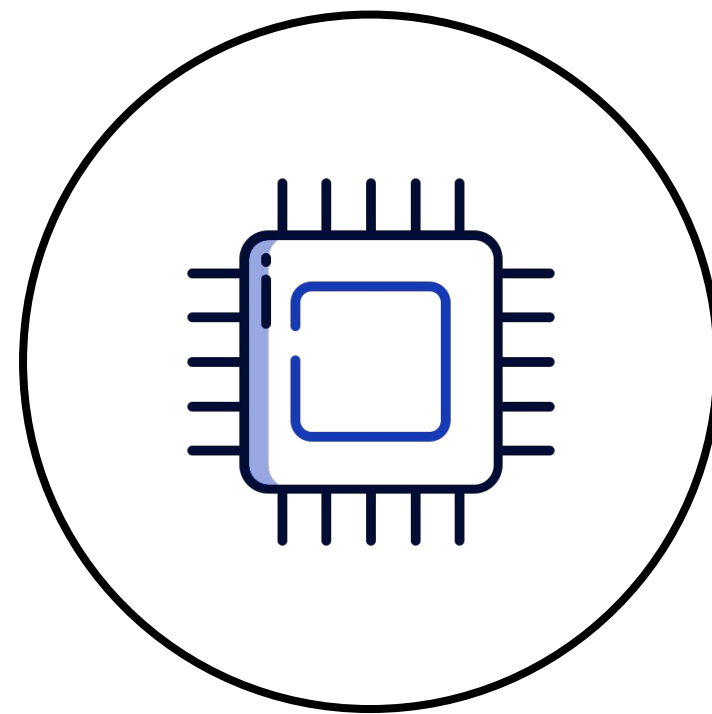
Полезные HTTP Headers



Authorization



Cache-Control

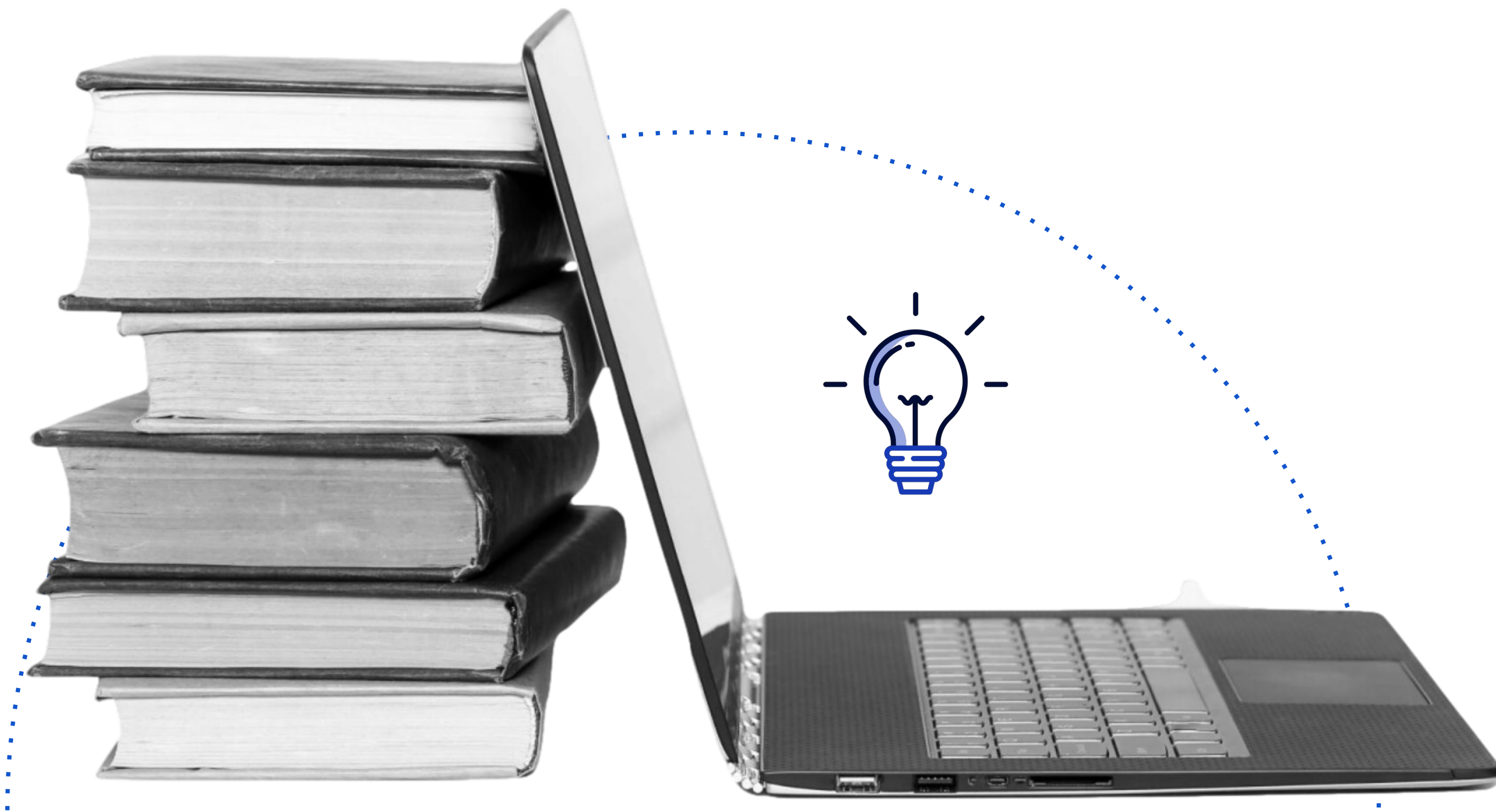


Origin



Content-Length

Примеры



REST API интернет-магазина

Ресурсы:



пользователь



заказ



товар



**картинка
товара**

REST API интернет-магазина

URI:

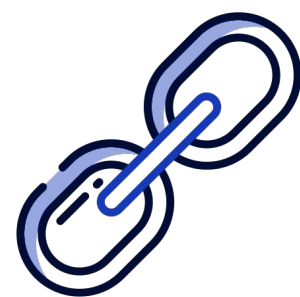
- ☆ **/user**, все пользователи
- ☆ **/user/:id**, конкретный пользователь
- ☆ **/order**, все заказы
- ☆ **/order/:id**, конкретный заказ
- ☆ **/product**, все товары
- ☆ **/product/:id**, конкретный продукт
- ☆ **/product/:id/image**, все картинки конкретного продукта
- ☆ **/product/:id/image/:id**, конкретная картинка конкретного продукта

REST API пример универсальности



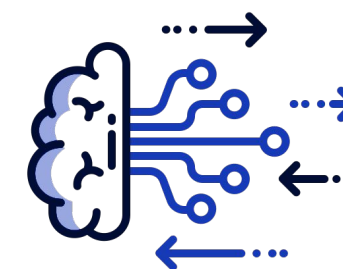
Ресурсы:

- ★ **хранилище данных**
 - ИМЯ
- ★ **файл**
 - ИМЯ
 - хранилище



URI:

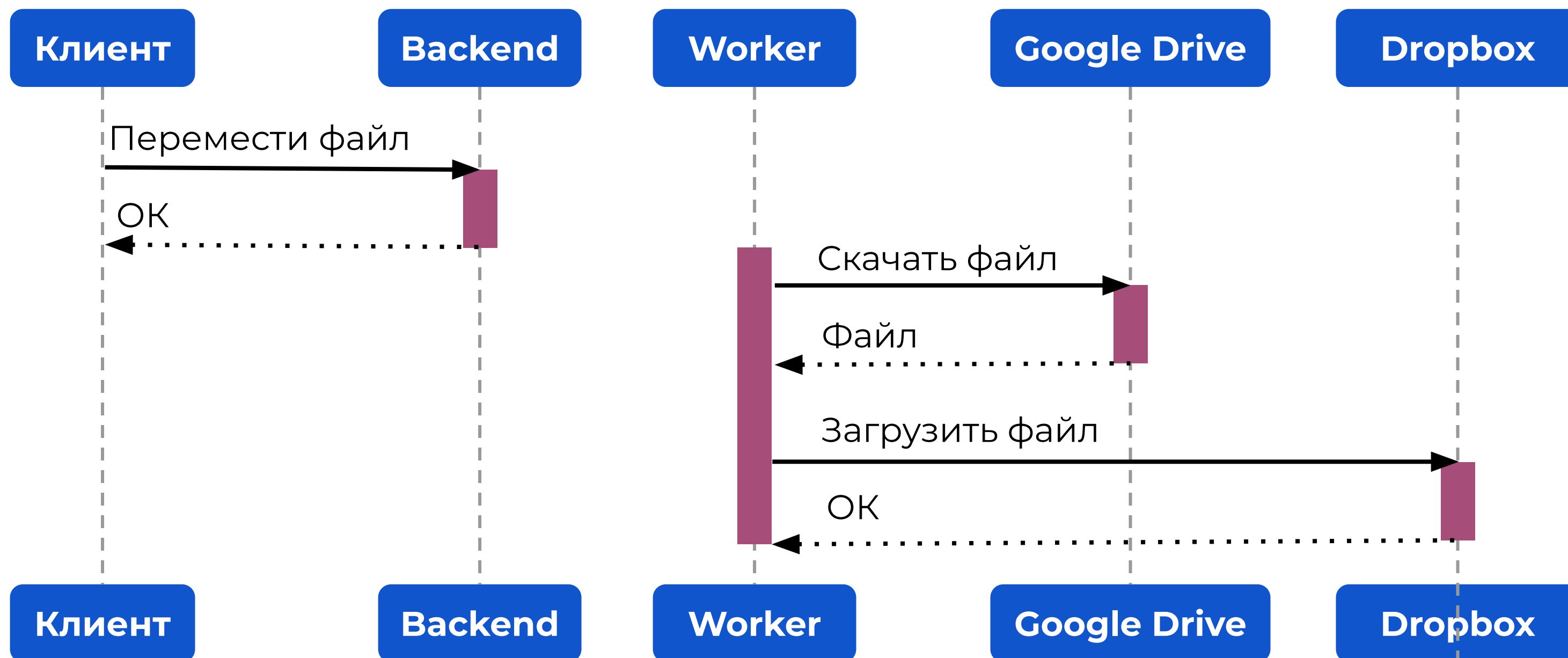
- ★ **`/file/:name`**
- ★ **`/storage/:name/file/:name`**



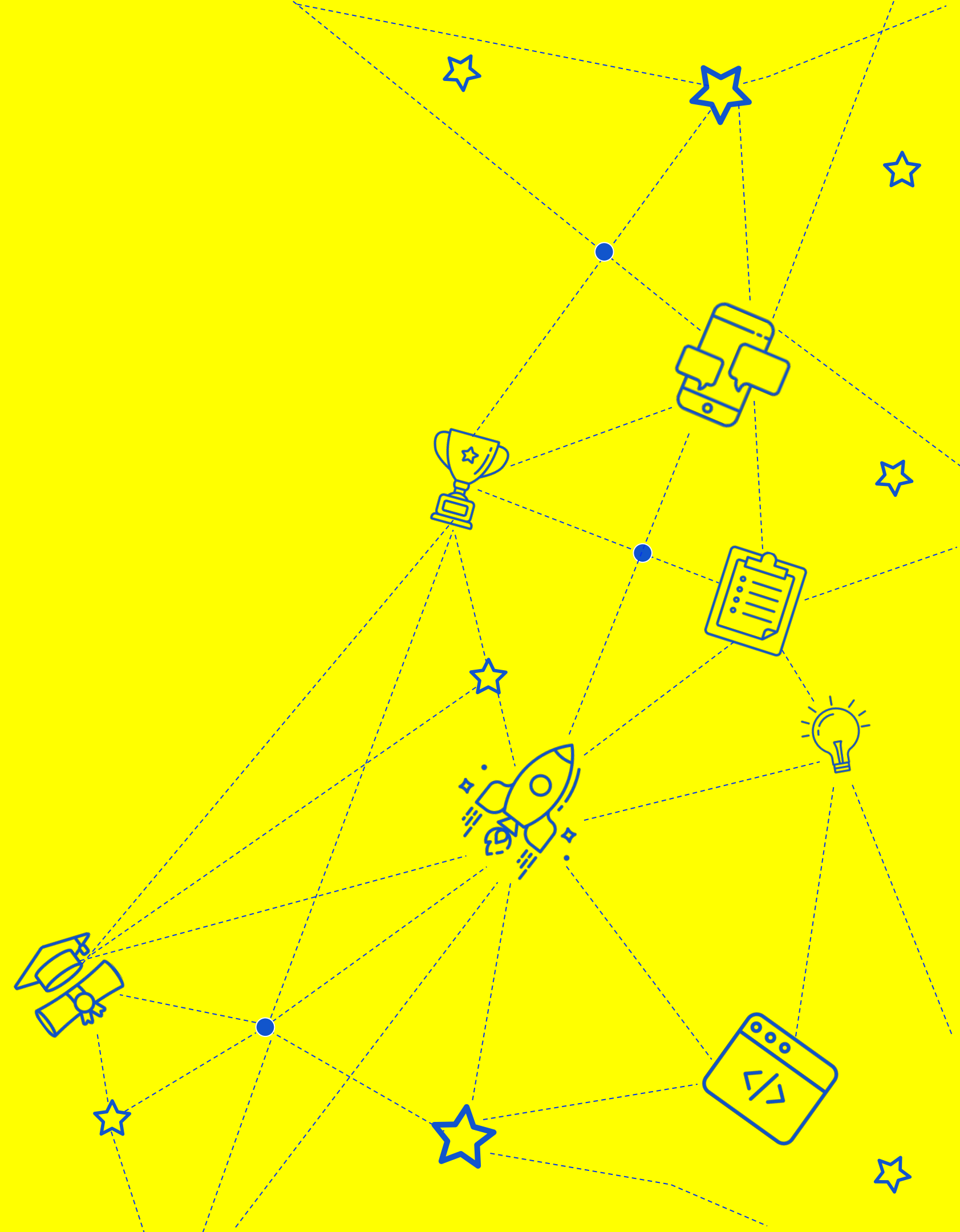
Сценарий:

Перемещение
файла в другое
хранилище

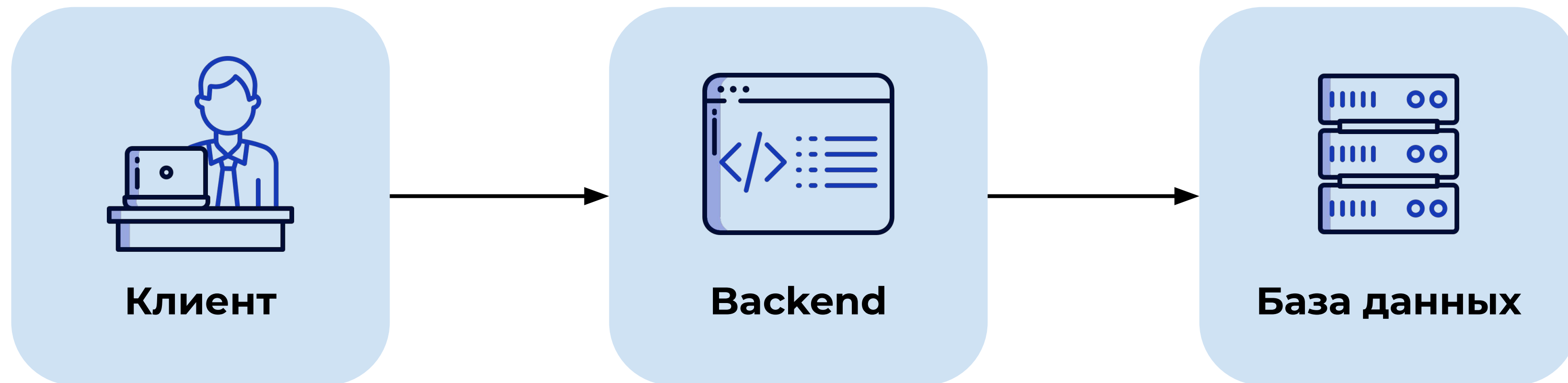
REST API пример универсальности



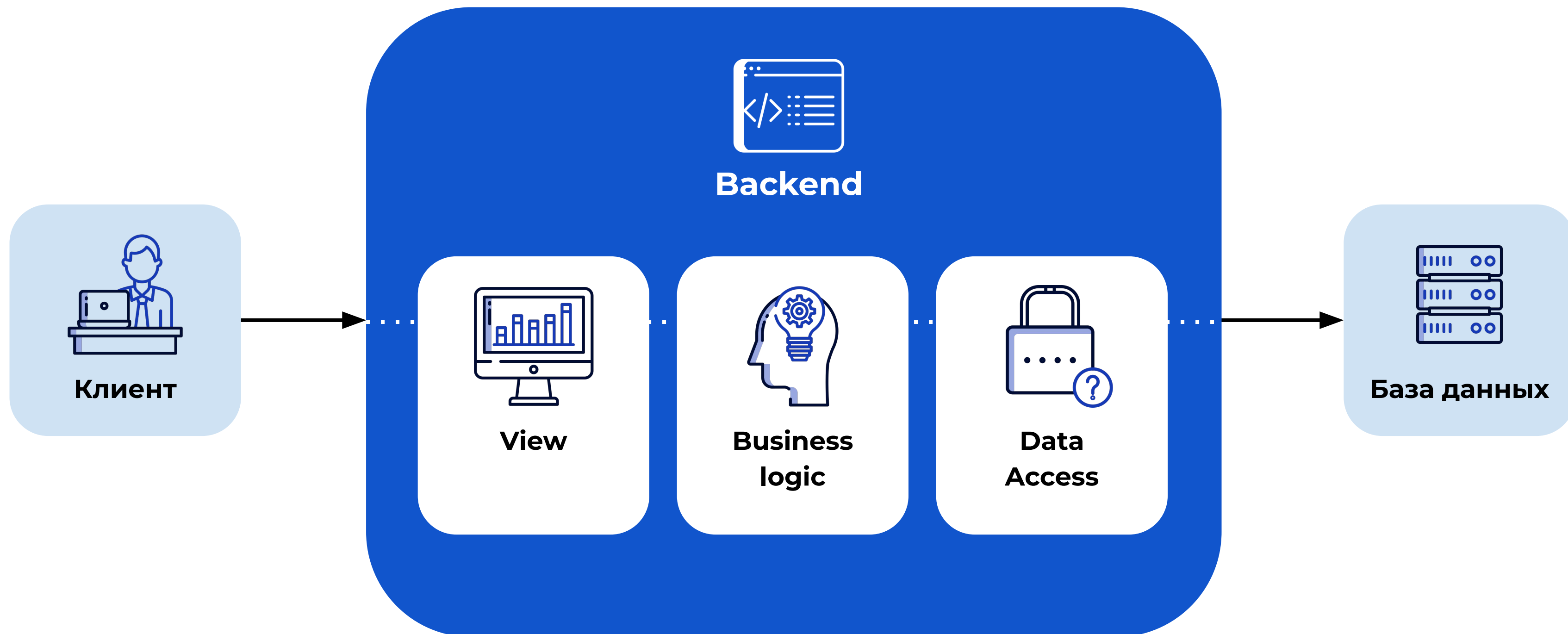
Паттерн контроллер и роутинг



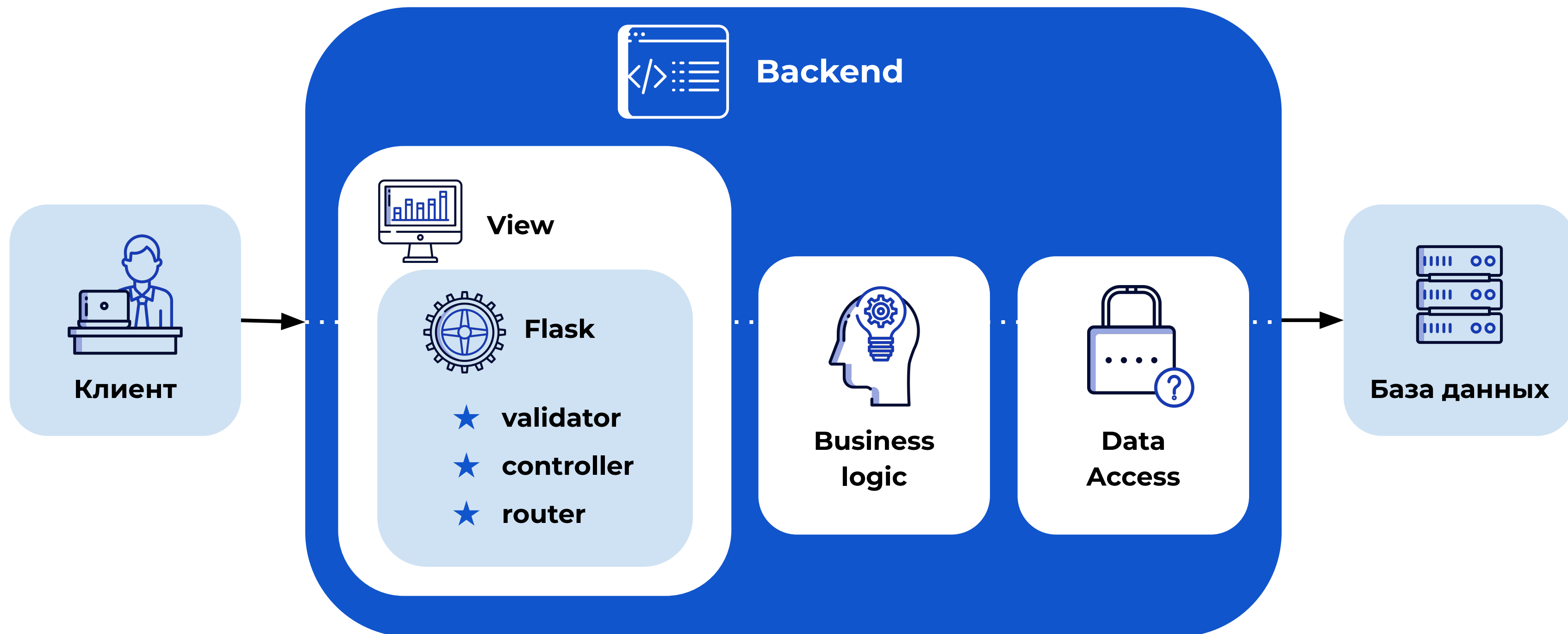
Как устроен backend



Как устроен backend

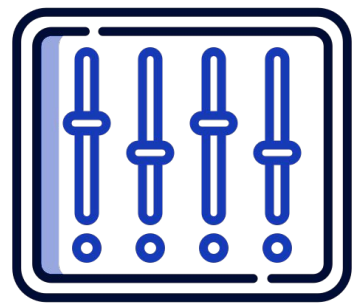


Как устроен backend



Что такое controller

Controller – адаптер между клиентом и бизнес-логикой.



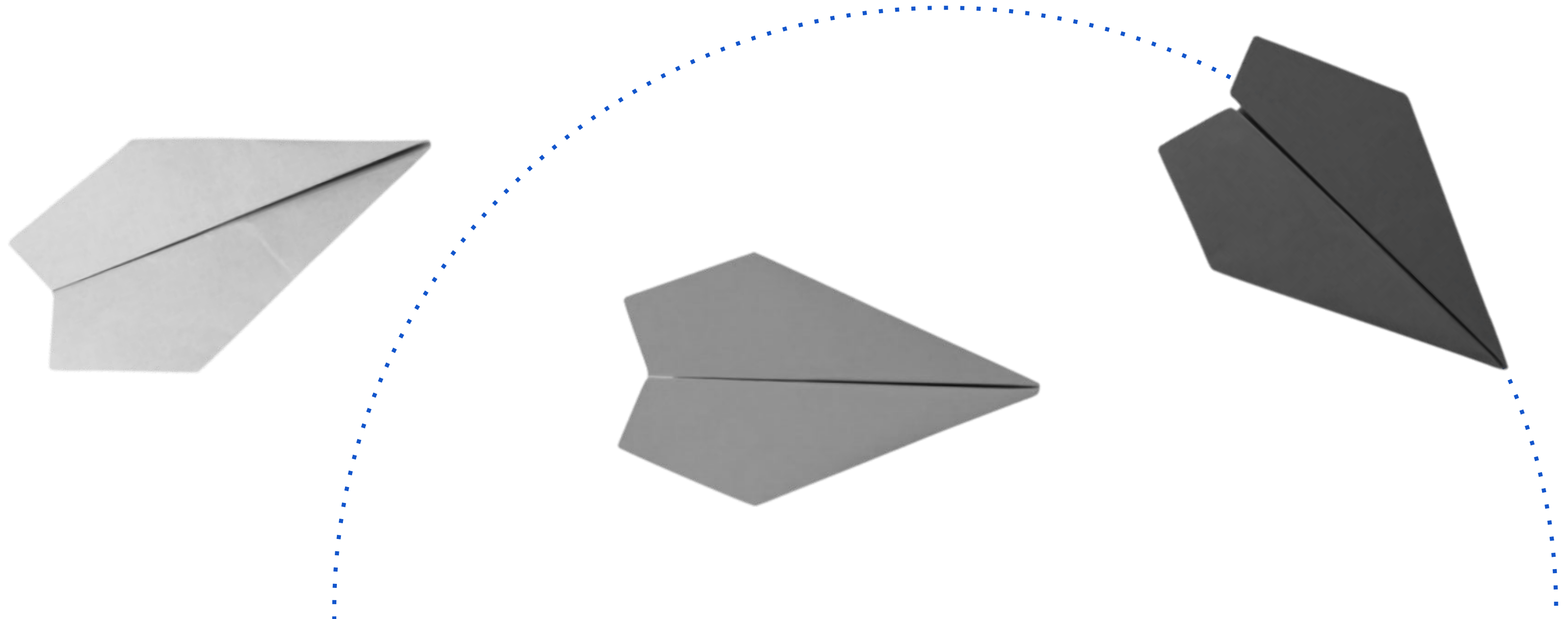
За что он отвечает:

- ★ аутентификацию
- ★ парсинг и валидацию входных данных
- ★ вызов соответствующей бизнес-логики
- ★ формирование, сериализацию и отправку ответа клиенту

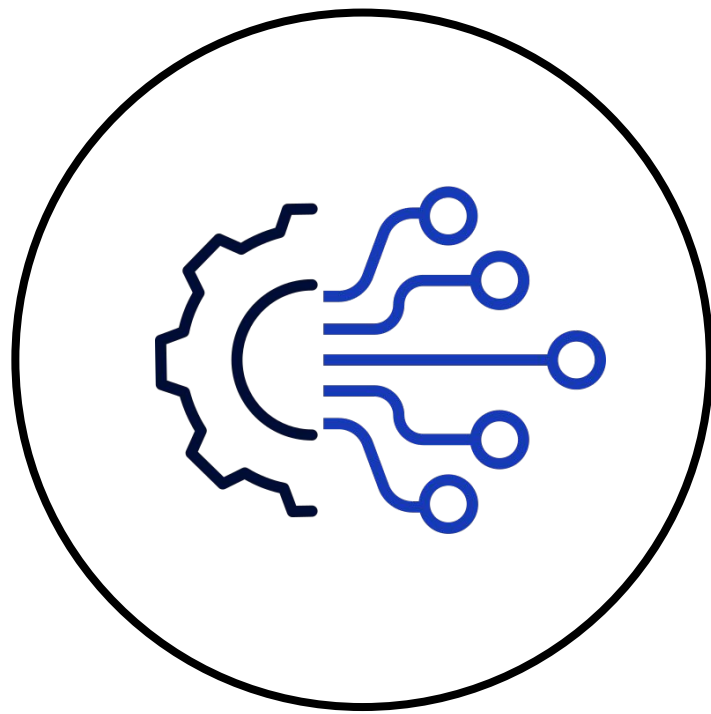


Что такое routing

Routing – процесс перенаправления запроса пользователя конкретному контроллеру.



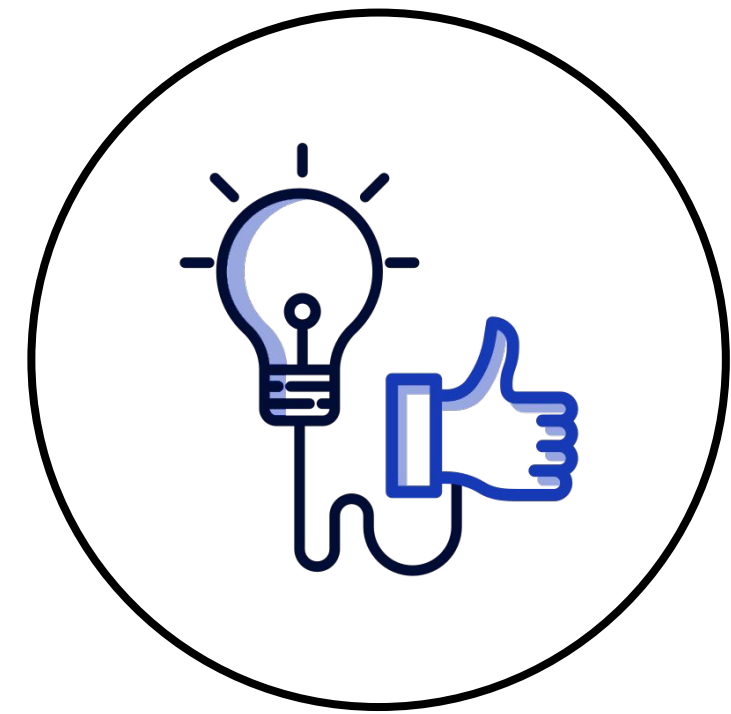
Чем мне поможет Flask



построить роутинг
между URI
и контроллерами

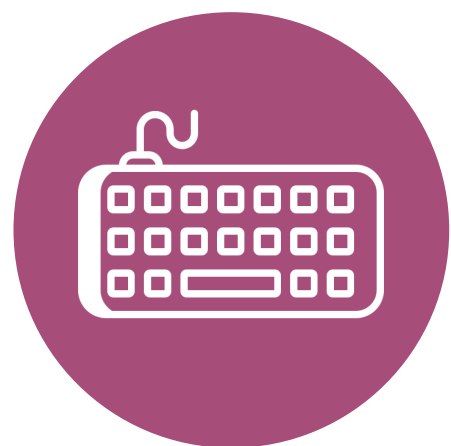


упростит парсинг
входных данных

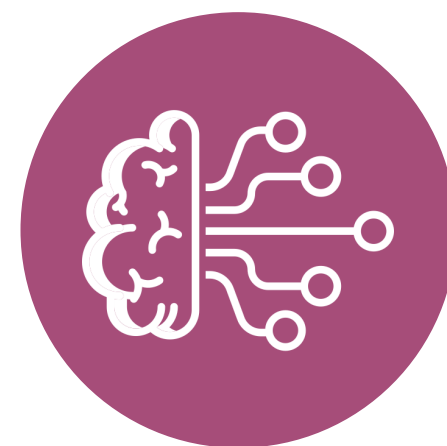


упростит
формирование
ответа клиенту

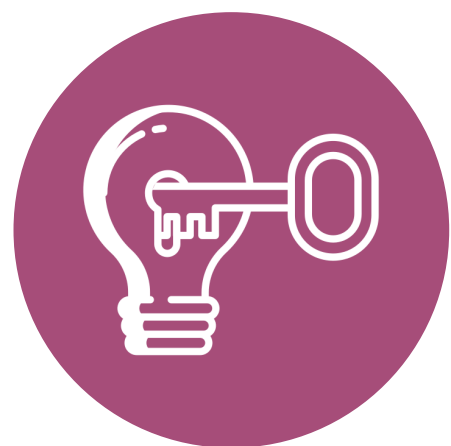
Типичные ошибки



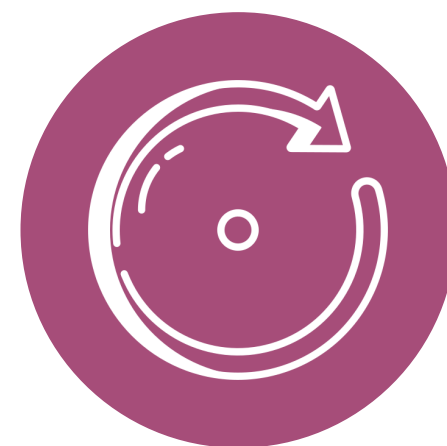
писать бизнес-логику
в контроллерах



не обрабатывать ошибки
бизнес-логики



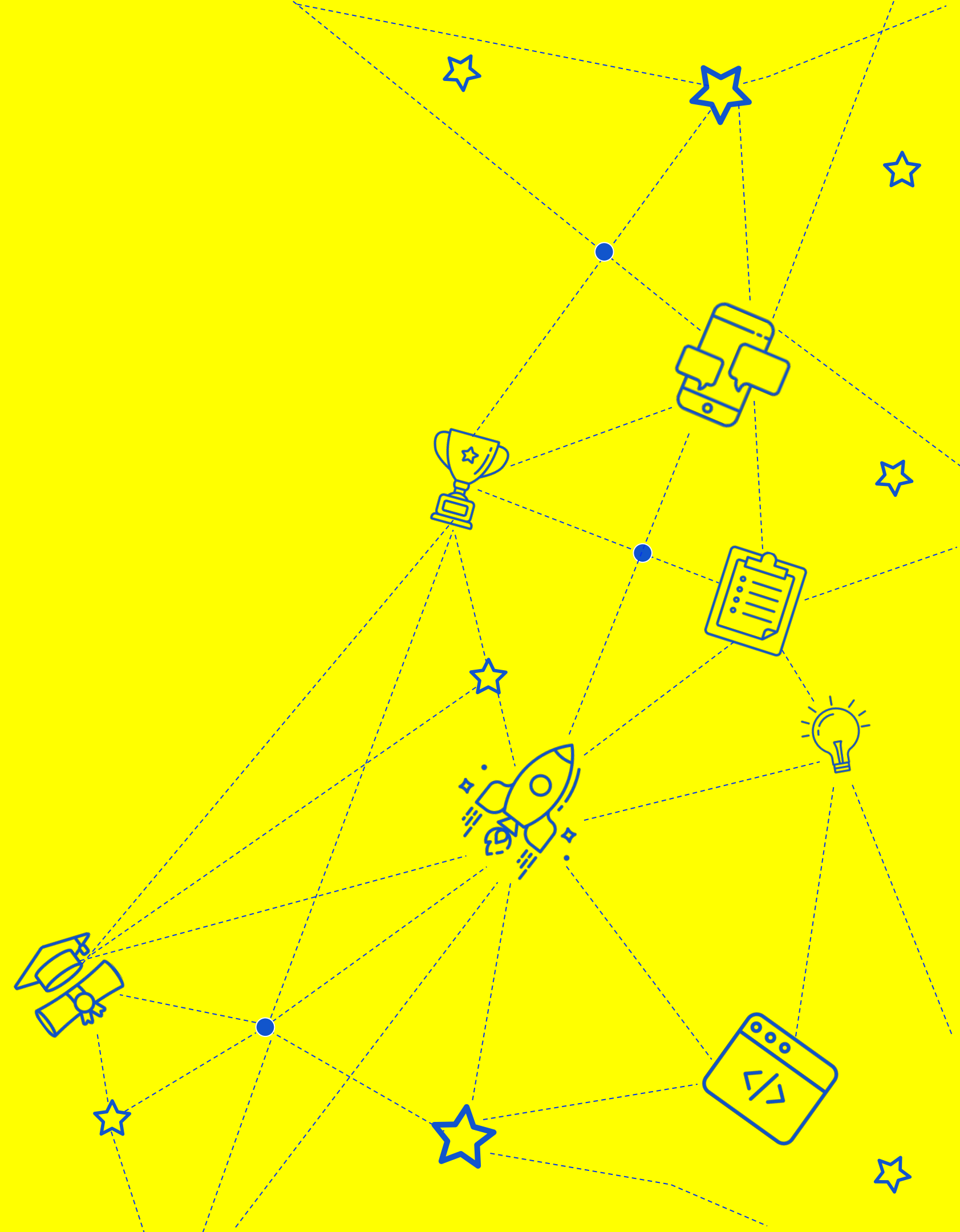
авторизация
в контроллере



отдавать в респонс то,
что вернула бизнес-логика

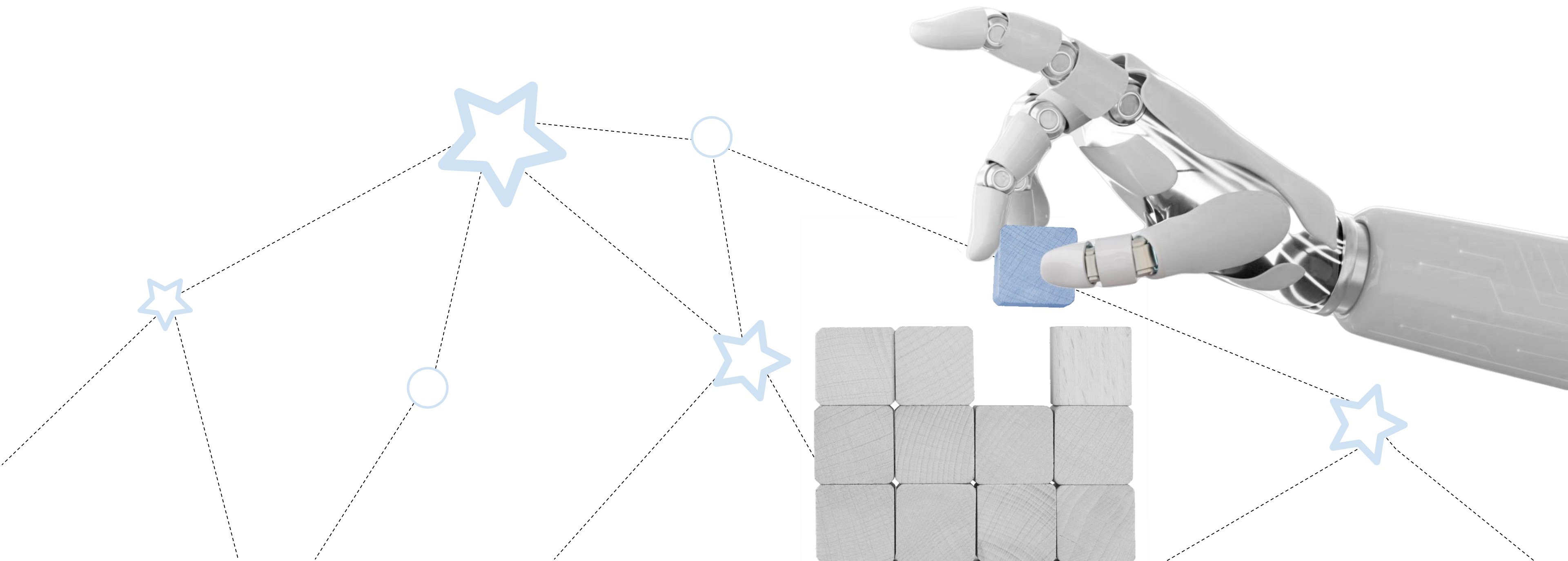
Flask

и с чем его едят



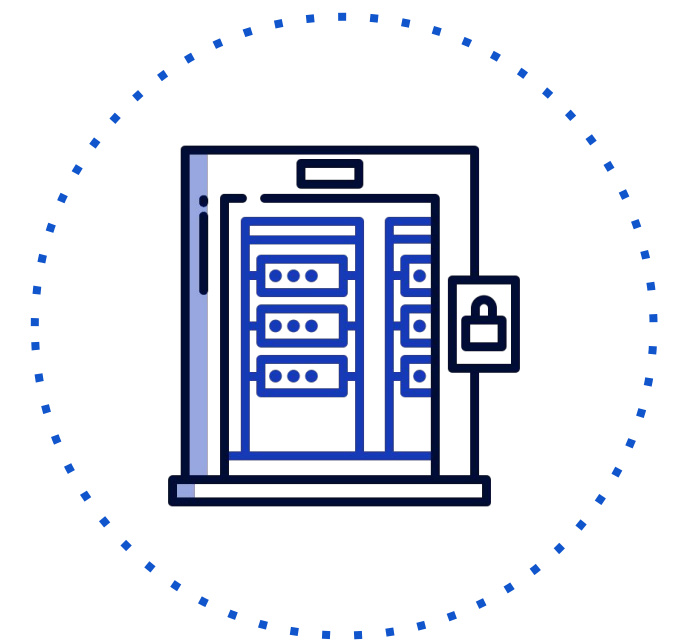
Что такое Flask

Flask – фреймворк для создания backend api.



Как создать сервер

```
from flask import Flask  
  
app = Flask(__name__)  
  
if __name__ == '__main__':  
    app.run(port=5000)
```



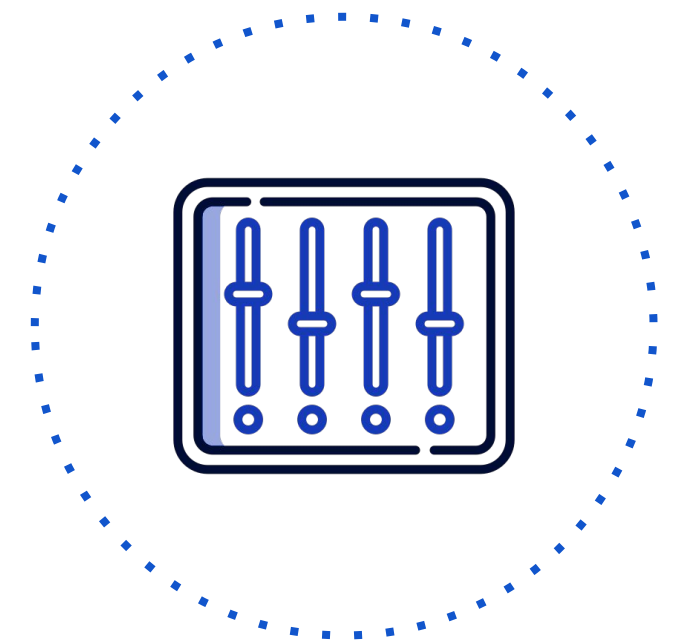
Как создать контроллер

```
from flask import Flask

app = Flask(__name__)

@app.route('/hello-world')
def hello_world():
    return 'Hello World'

if __name__ == '__main__':
    app.run(port=5000)
```



Вы умеете создавать api на flask

Directed by
ROBERT B. WEIDE

URL параметры

```
@app.route('/hello/<name>')
def hello_by_name(name: str):
    return f'Hello {name}'

@app.route('/blog/<int:id>')
def show_blog(id: int):
    return f'Blog Number {id}'

@app.route('/temperature/<float:degrees>')
def temperature(degrees: float):
    return f'You temperature is {degrees}'

@app.route('/proxy/<path:route>')
def proxy(route: str):
    return f'Let\'s proxy to: {route}'
```



HTTP methods

```
@app.route('/request, methods = ['POST', 'GET'])
def login():
    if request.method == 'POST':
        return 'This is POST'
    else:
        return 'This is GET'
```



HTTP methods

```
@app.get('/get-request')
def get():
    return 'This is GET'

@app.post('/post-request')
def post():
    return 'This is POST'

@app.delete('/post-request')
def delete():
    return 'This is DELETE'

@app.patch('/patch-request')
def patch():
    return 'This is PATCH'
```



GET params

Запрос:

GET /order?page=1&limit=10

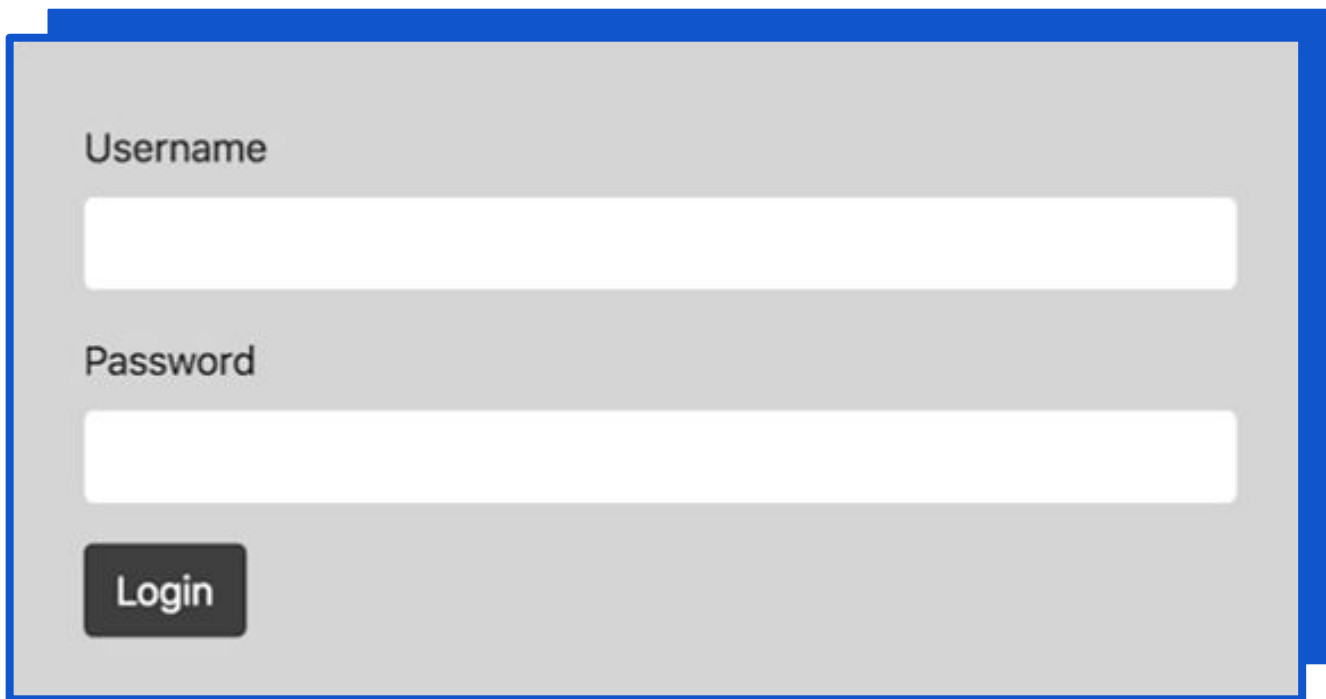
```
from flask import Flask, request
app = Flask(__name__)

@app.get('/order')
def order():
    limit = request.args.get('limit')
    page = request.args.get('page')
    return f'Getting orders from page {page} with limit {limit}'

app.run()
```

Формы

Форма на сайте



A login form with a light gray background and a blue border. It contains two input fields: the first is labeled 'Username' and the second is labeled 'Password'. Below the password field is a dark gray button with the text 'Login' in white.

Код

```
from flask import Flask, request
app = Flask(__name__)

@app.post('/login')
def login():
    name = request.form.get('name')
    password = request.form.get('password')
    # логика аутентификации
    return 'ok'

app.run()
```

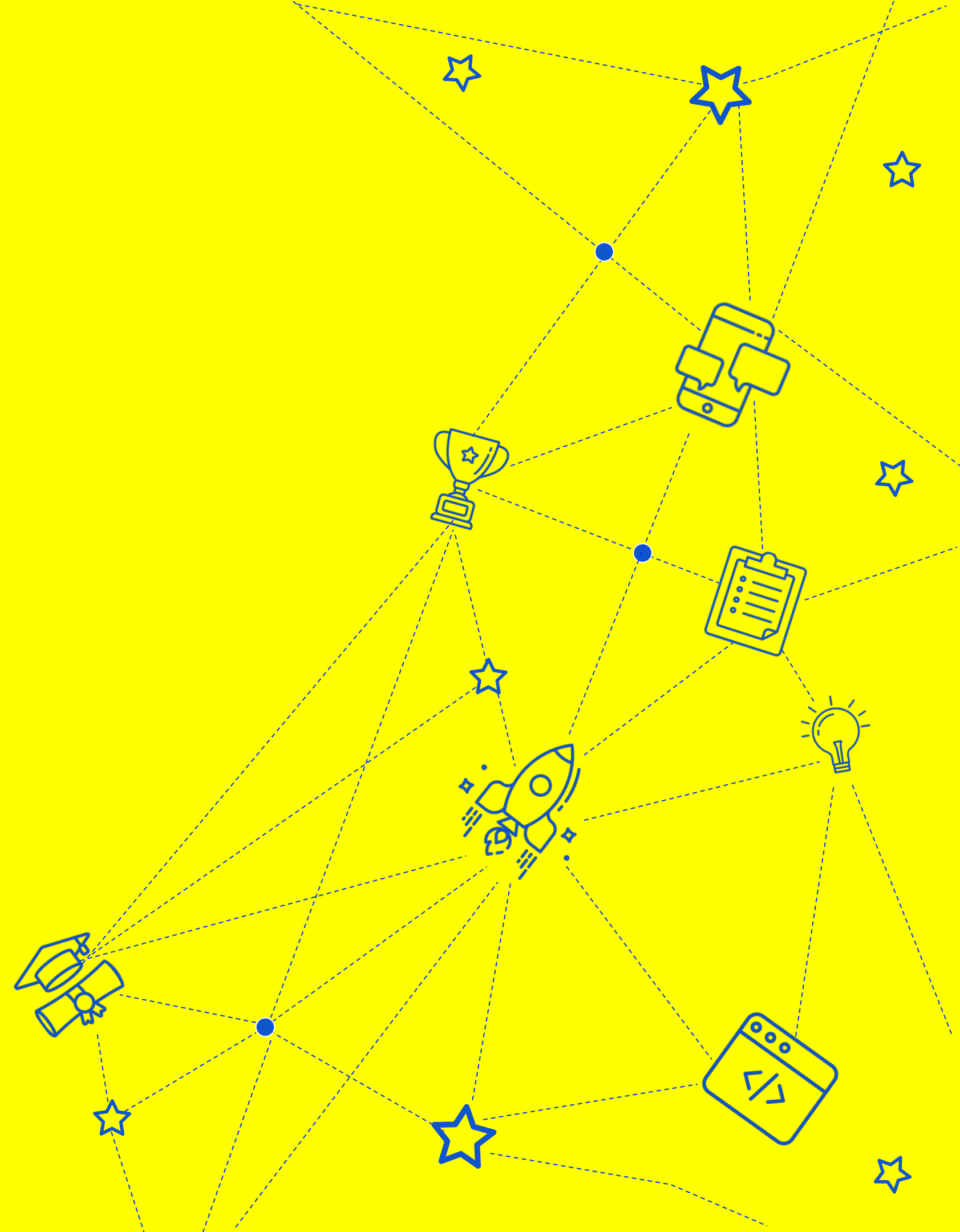
Response

```
@app.get("/me")
def get_me():
    return {
        "username": "Jon Show",
    }
```

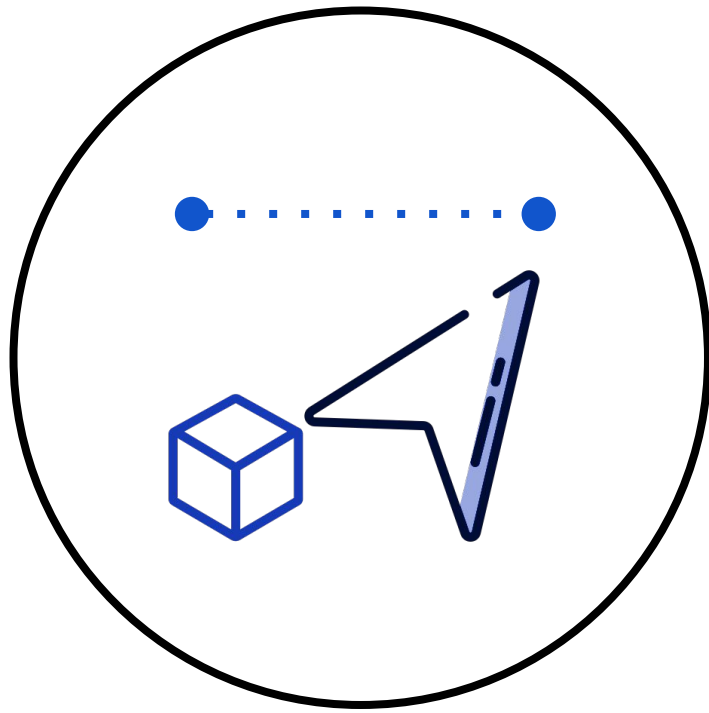
```
@app.get("/user")
def get_user():
    return "Forbidden", 403
```



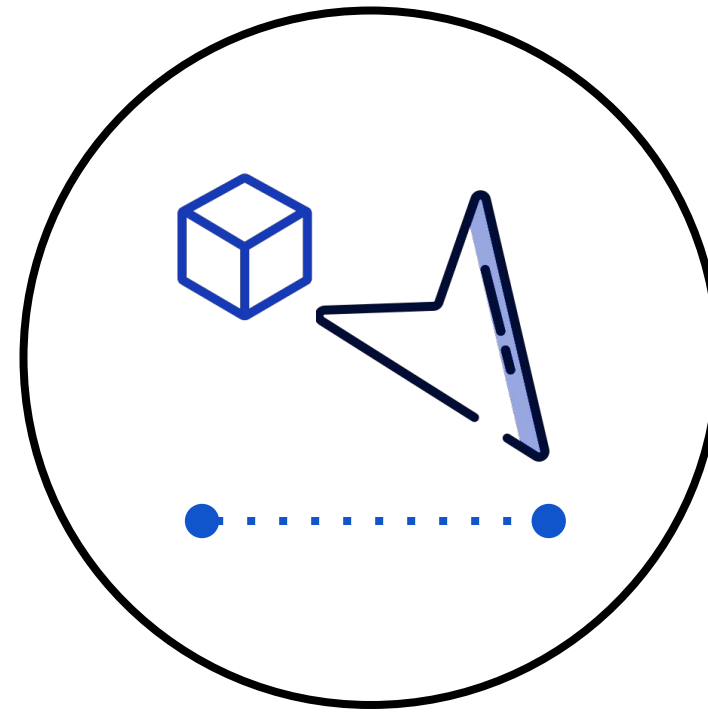
Сериализация и десериализация



Основные понятия



Сериализация – процесс превращения объекта в строку определенного формата.



Десериализация – обратный процесс, превращение строки определенного формата в объект.

Основные форматы

QueryString

JSON

XML



Flask из коробки

```
from flask import Flask, request
app = Flask(__name__)

@app.get('/me')
def get_me():
    name = request.json.get('name')
    return {
        "username": name,
    }

app.run()
```



**А что, если нам
нужно обработать
сложный случай?**

marshmallow



Сериализация

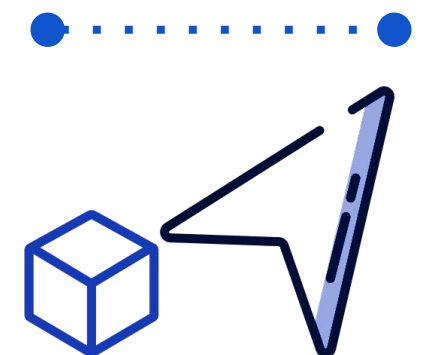
```
from datetime import date
from marshmallow import Schema, fields

class ProductSchema(Schema):
    name = fields.Str()

class OrderSchema(Schema):
    deliver_at = fields.Date()
    products = fields.Nested(ProductSchema(many=True))

book = dict(name="Book")
album = dict(products=[book], deliver_at=date(2022, 12, 17))

schema = OrderSchema()
result = schema.dump(album)
print(result)
# {'products': [{'name': 'Book'}], 'deliver_at': '2022-12-17'}
```



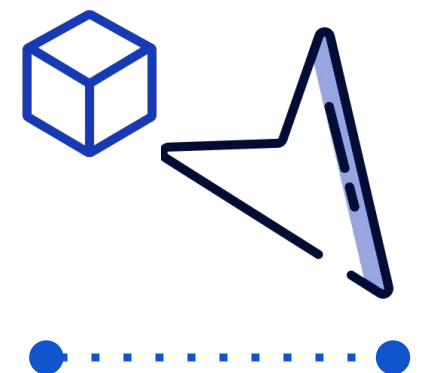
Десериализация

```
from marshmallow import Schema, fields

class UserSchema(Schema):
    name = fields.Str()
    email = fields.Email()
    created_at = fields.DateTime()

user_data = {
    "created_at": "2014-08-11T05:26:03.869245",
    "email": "ken@yahoo.com",
    "name": "Ken",
}

schema = UserSchema()
result = schema.load(user_data)
print(result)
# {'name': 'Ken',
#  'email': 'ken@yahoo.com',
#  'created_at': datetime.datetime(2014, 8, 11, 5, 26, 3, 869245)},
```



Валидация

```
from marshmallow import Schema, fields, validate, ValidationError

class UserSchema(Schema):
    name = fields.Str(validate=validate.Length(min=1))
    permission = fields.Str(validate=validate.OneOf(["read", "write", "admin"]))
    age = fields.Int(validate=validate.Range(min=18, max=40))

in_data = {"name": "", "permission": "invalid", "age": 71}
try:
    UserSchema().load(in_data)
except ValidationError as err:
    print(err.messages)
# {'age': ['Must be greater than or equal to 18 and less than or equal to 40.'],
#  'name': ['Shorter than minimum length 1.'],
#  'permission': ['Must be one of: read, write, admin.']}
```


Интеграция с flask

```
from flask import Flask
from flask_marshmallow import Marshmallow

app = Flask(__name__)
ma = Marshmallow(app)
```



Интеграция с flask

```
from your_orm import Model, Column, Integer, String, DateTime

class User(Model):
    email = Column(String)
    password = Column(String)
    date_created = Column(DateTime, auto_now_add=True)

#####
# Файл объявления схемы сериализации

class UserSchema(ma.Schema):
    class Meta:
        # Fields to expose
        fields = ("email", "date_created")

user_schema = UserSchema()
users_schema = UserSchema(many=True)
```

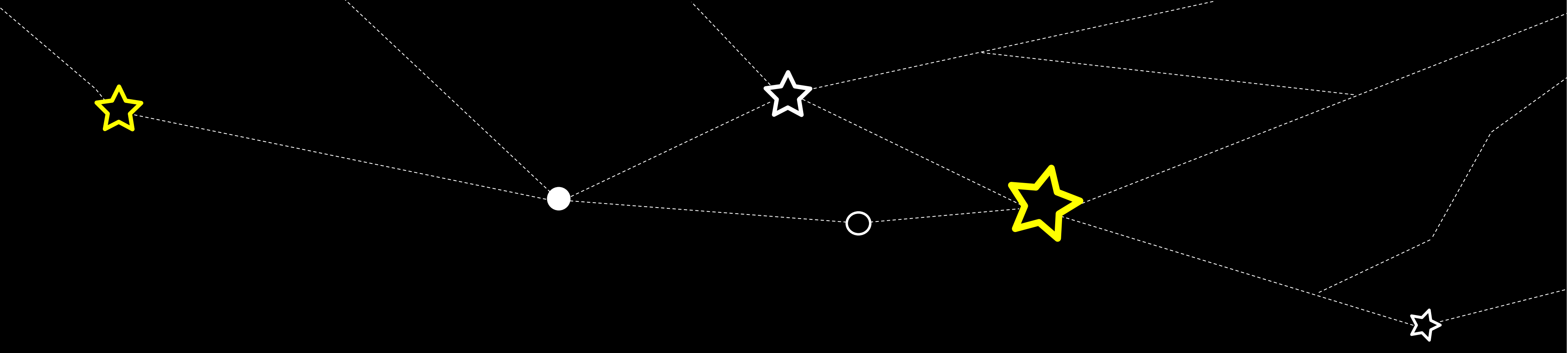


Интеграция с flask

```
@app.route("/api/users/")
def users():
    all_users = User.all()
    return users_schema.dump(all_users)
```

```
@app.route("/api/users/<id>")
def user_detail(id):
    user = User.get(id)
    return user_schema.dump(user)
```





СПАСИБО ЗА ВНИМАНИЕ

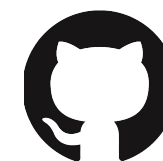
Игорь Кривонос



<https://www.linkedin.com/in/ihar-kryvanos-7066b886/>



@ikryvanos



<https://github.com/ikryvanos>